As mecânicas do nosso jogo

O nome, o que cada uma faz e como o código funciona.

24 mecânicas e sistemas de suporte explicados com exemplos do projeto, termos usados em jogos e pseudocódigo em linguagem simples.

01 DESENVOLVER
Construir e obter recursos

02 PREPARAR
Recrutar e distribuir equipes

03 ENFRENTAR
Defender portões e explorar

04 EXPANDIR
Conquistar e sustentar regiões

Análise do projeto local em 10 de novembro de 2026

Preparado por Lain para Gustavo

VISÃO GERAL

Como ler este guia

O QUE FOI ANALISADO

A análise parte dos scripts C# atuais e das configurações salvas nas cenas. Os nomes em inglês são termos usuais ou descrições equivalentes; não pressupõem que o projeto copie exatamente a implementação de outro jogo.

DOIS COMBATES, DUAS REGRAS

Portões: soldados em quantidade, defesa efetiva e resultado automático por cálculo.
Exploração: aventureiros individuais, formação, vida e ações automáticas por iniciativa. Os dois sistemas não compartilham a mesma fórmula de batalha.

ROTEIRO DE LEITURA

03	Controle e interação	01-02
04	Economia da cidade	03-04
05	Recrutamento	05-06
06	Defesa: tropas e ondas	07-08
07	Defesa: resultados e avisos	09-10
08	Exploração e desbloqueios	11-12
09	Iniciativa e formação	13-14
10	Atributos, encontros e saque	15-16
11	Produção e territórios	17-18
12	Inventário e relatório	19-20
13	Câmera e profundidade	21-22
14	Cenas e salvamento	23-24
15	Relógios e organização do código	
16	Vocabulário e limites atuais	

Sobre os exemplos. Os quadros são pseudocódigo: resumem a lógica e não devem ser colados diretamente no C#. Os valores indicados são da revisão analisada e podem mudar no Inspector ou nos assets.

Como localizar os arquivos. As referências das fichas são relativas à pasta **Assets** do projeto **TCC-Game**. Pastas chamadas Prototypes ainda contêm componentes usados no jogo.

FICHAS 01 E 02

Controle e interação

01 Movimento com visão superior

Top-down movement

O que faz. Permite explorar a cidade em duas dimensões. A animação acompanha a direção em que o corpo realmente se move e mantém a última direção ao parar.

Como foi implementado. GameInput lê a ação Player/Move. moviment guarda a direção em Update e aplica a velocidade ao Rigidbody2D em FixedUpdate. Os limites consideram o tamanho do colisor. PlayerAnimationController escolhe andar/parado e espelha a arte para a esquerda.

```
direcao = lerEntrada("Player/Move")
velocidadeDoCorpo = direcao * velocidadeConfigurada
animacao = escolherEstado(velocidadeDoCorpo)
```

Para entender: A física move o personagem; a animação apenas troca sua aparência, com Root Motion desativado.

No projeto: Scripts/Player/moviment.cs; Scripts/Player/PlayerAnimationController.cs; Scripts/Core/Input/GameInput.cs

02 Interação por proximidade

Contextual interaction

O que faz. Escolhe o NPC ou objeto próximo que deve responder ao comando de interação. Evita abrir várias ações ao mesmo tempo quando existem alvos vizinhos.

Como foi implementado. PlayerInteractor registra contatos de colisão e de trigger, filtra quem pode interagir e escolhe o mais próximo. Todos seguem o contrato IInteractable. Ao abrir um painel, um bloqueio impede o movimento até que os donos desse bloqueio o liberem.

```
alvo = interagivelValidoMaisProximo()
se apertouInteragir e naoHaModal: alvo.Interact(player)
movimentoBloqueado = quantidadeDeBloqueios > 0
```

Para entender: O mesmo comando pode abrir um prédio, conversar com um NPC ou inspecionar um aventureiro.

No projeto: Scripts/Core/Input/PlayerInteractor.cs; Scripts/Core/Input/IInteractable.cs

FICHAS 03 E 04

Economia da cidade

03 Construção e melhorias

Building upgrades / resource gating

O que faz. Converte ouro e materiais em níveis de edifícios. As melhorias aumentam benefícios e capacidades, conectando a exploração ao desenvolvimento da cidade.

Como foi implementado. BuildingDefinition guarda preços, requisitos e limites. TryUpgrade valida tudo antes de descontar os custos e aumentar o nível. Se a gravação falhar, restaura os valores. Depois do sucesso, LevelChanged avisa outros sistemas para atualizar seus benefícios.

```
se nivel == limite: recusarMelhoria()
se faltamOuroOuMateriais: recusarMelhoria()
consumirCustos(); nivel += 1; salvar(); avisarMudanca()
```

Para entender: Quando não existe um preço específico por nível, o custo usa base × multiplicador elevado ao nível atual.

No projeto: Scripts/City/BuildingDefinition.cs; Scripts/City/BuildingCalculator.cs; Scripts/City/BuildingService.cs

---

04 Renda passiva e impostos

Passive income / timed accrual

O que faz. O Mercado Local acumula dinheiro ao longo do tempo. O jogador precisa recolher os impostos para transformar o valor acumulado em saldo utilizável.

Como foi implementado. TaxService compara o horário atual com o último cálculo. TaxCalculator converte a produção por minuto em renda pelos segundos elegíveis. Há um limite de 24 horas de acumulação entre coletas. A coleta transfere a parte inteira e preserva a fração restante.

```
segundos = limitarTempoElegivelAoTetoDe24Horas()
acumulado += producaoPorMinuto * segundos / 60
ouro += parteInteira(acumulado)
```

Para entender: Ao melhorar o mercado, o tempo anterior é calculado com a produção antiga, evitando aplicar a taxa nova retroativamente.

No projeto: Scripts/City/TaxAccrualService.cs; Scripts/City/TaxCalculator.cs

FICHAS 05 E 06

Recrutamento

05 Recrutamento de soldados

Unit recruitment / restocking

O que faz. Compra guerreiros, arqueiros e magos para a reserva militar. Cada tipo possui preço e estoque; comprar reduz o estoque e aumenta o exército disponível.

Como foi implementado. RecruitmentService valida quantidade, estoque e dinheiro. A compra e a gravação formam uma operação com restauração em caso de falha. O estoque é reposto a cada seis horas de relógio real; a capacidade da reposição depende do prédio militar correspondente.

```
custo = precoUnitario * quantidade
validarEstoqueESaldo(); descontarOuro()
estoque -= quantidade; reserva += quantidade; salvar()
```

Para entender: Preços atuais: guerreiro 3, arqueiro 5 e mago 8. Capacidade de reposição: piso de 10 × (1 + 0,30 × nível).

No projeto: Scripts/Portao/RecruitmentService.cs

---

06 Ofertas de aventureiros

Rotating roster / hero recruitment

O que faz. A taverna apresenta ofertas temporárias de personagens individuais. É possível contratar mais de um aventureiro da mesma classe e dar um nome a cada um.

Como foi implementado. TavernRecruitmentWaveService embaralha o catálogo em ciclos e salva as ofertas. O spawner instancia os prefabs. A compra cria um instanceId próprio, desconta o preço e consome só aquela oferta. Uma nova rodada depende do prazo vencido e de entrar novamente na taverna.

```
ofertas = consultarOuSortearRodadaAoEntrar()
personagem = criarRegistroComIdUnico(classe, nome)
pagar(); salvarPersonagem(); consumirUmaOferta()
```

Para entender: A cena Taverna está configurada com 10 s por rodada e 2 ofertas no nível 1, mais 1 por nível adicional.

No projeto: Scripts/Portao/TavernRecruitmentWaveService.cs; Scripts/Aventureiro/TavernRecruitableSpawner.cs; Scripts/Core/PlayerInventoryService.cs

FICHAS 07 E 08

Defesa: tropas e ondas

07 Deslocamento de tropas

Troop deployment / travel time

O que faz. Redistribui soldados entre reserva e portões. O tempo de viagem torna importante preparar a defesa antes de a onda chegar.

Como foi implementado. Dispatch valida a rota e a quantidade, retira as tropas da origem e registra uma transferência com horário de chegada. ResolveArrivals entrega as tropas ao destino quando o relógio estratégico alcança esse horário. O intervalo padrão de viagem é de 15 a 25 segundos estratégicos.

```
origem -= quantidade
emTransito.adicionar(destino, quantidade, horarioChegada)
se tempoEstrategico >= chegada: destino += quantidade
```

Para entender: Soldados em trânsito não aparecem simultaneamente na origem e no destino. O registro da viagem permanece no save.

No projeto: Scripts/Campaign/TroopDeploymentService.cs; Scripts/Campaign/CampaignRules.cs

---

08 Defesa por ondas e escalada

Wave-based defense / difficulty scaling

O que faz. Organiza ataques sucessivos aos portões. A ameaça cresce ao longo da campanha e a composição dos inimigos ganha variedade.

Como foi implementado. WaveGenerator escolhe portão, prazo e composição usando uma semente de aleatoriedade. CampaignRules calcula um orçamento de ameaça pela onda e aplica um ajuste limitado pelo progresso militar. O gerador gasta esse orçamento em inimigos e impede três ondas seguidas no mesmo portão.

```
ameacaBase = arredondar(10 * 1.22^(onda - 1))
ameaca = ameacaBase * fatorDeProgressoLimitado
sortearPortao(); preencherInimigosAteGastarOrcamento()
```

Para entender: A campanha prevê 10 ondas; a primeira tem 180 s de preparação. Brutos entram a partir da terceira e xamãs da quinta.

No projeto: Scripts/Campaign/WaveGenerator.cs; Scripts/Campaign/WaveDirectorService.cs; Scripts/Campaign/CampaignRules.cs

FICHAS 09 E 10

Defesa: resultados e avisos

09 Resolução estratégica e vantagens

Auto-resolve / unit counters

O que faz. Resolve o confronto do portão por números: força da guarnição, composição adversária, baixas e dano à estrutura. É diferente da batalha individual de exploração.

Como foi implementado. ThreatForecastService calcula a defesa efetiva, ponderando vantagens contra os inimigos presentes. BattleCalculator calcula pressão, perdas e dano ao portão. BattleResolutionService aplica e salva o resultado uma vez. Um portão rompido causa derrota; sobreviver à décima onda conclui a campanha com vitória.

```
defesa = somarPoderDasTropasComVantagens()
pressao = ameaca / maximo(1, defesa)
aplicarBaixasEDano(); registrarResultado(); salvar()
```

Para entender: Guerreiros favorecem o combate contra saqueadores; arqueiros, contra xamãs; magos, contra brutos. Mesmo uma defesa forte pode sofrer baixas.

No projeto: Scripts/Campaign/ThreatForecastService.cs; Scripts/Campaign/BattleCalculator.cs; Scripts/Campaign/BattleResolutionService.cs

10 Antecipação de ameaça

Threat telegraphing / combat forecast

O que faz. Avisa que um ataque está chegando e apresenta uma estimativa da situação da defesa. Dá ao jogador tempo e informação para decidir onde enviar tropas.

Como foi implementado. WaveDirectorService compara o prazo com o tempo estratégico e avança os avisos em 90, 30 e 10 segundos restantes. O prognóstico divide defesa efetiva por ameaça. A interface traduz esse número em favorável, arriscado ou desfavorável.

```
restante = chegadaDaOnda - tempoEstrategico
razao = defesaEfetiva / ameaca
razao >= 1.15: favoravel; razao < 0.85: desfavoravel
```

Para entender: O intervalo de 0,85 até menos de 1,15 é arriscado. A indicação não significa ausência de baixas nem uma rolagem percentual de vitória.

No projeto: Scripts/Campaign/WaveDirectorService.cs; Scripts/Campaign/ThreatForecastService.cs; Scripts/Campaign/ThreatPresentationService.cs

FICHAS 11 E 12

Exploração e desbloqueios

11 Expedições e composição de grupo

Party management / mission state machine

O que faz. Envia de um a três aventureiros para capturar ou defender uma região. Um personagem ocupado não pode participar de outra missão ou guarda ao mesmo tempo.

Como foi implementado. TrySendParty verifica contratação, destino, tamanho e disponibilidade. Cria uma cópia dos atributos para a missão, chamada snapshot, e registra a chegada. Os estados controlam o fluxo: viajando, pronto para batalha, em batalha, retornando e retornado; derrota e retirada são saídas alternativas.

```
validarGrupoEDestino()
missao.party = copiarAtributosDosSelecionados()
missao.status = Travelling; chegada = agora + 5
```

Para entender: Há uma expedição ativa por vez. A cópia dos atributos protege uma viagem já iniciada de alterações posteriores na definição do personagem.

No projeto: Scripts/Exploration/ExplorationService.cs; Scripts/Exploration/ExplorationTypes.cs

---

12 Desbloqueio de regiões

Progression gating / prerequisite graph

O que faz. Libera destinos conforme as conquistas anteriores. Algumas regiões exigem uma vitória prévia; outras exigem mais de uma.

Como foi implementado. ExplorationProgression mantém um dicionário: cada destino aponta para seus pré-requisitos. IsUnlocked só retorna verdadeiro quando todos aparecem na lista de regiões concluídas. A mesma regra é consultada pela seleção no mapa e pela validação do envio.

```
para cada requisito do destino:
    se requisito naoEstaConcluido: retornar bloqueado
retornar liberado
```

Para entender: Reino Real exige Floresta Esmeralda e Montanhas Gélidas. Colinas do Vento, Floresta das Sombras e Caverna do Gelo começam acessíveis.

No projeto: Scripts/Exploration/ExplorationProgression.cs; Scripts/Exploration/ExplorationService.cs

FICHAS 13 E 14

Iniciativa e formação

13 Combate automático por iniciativa

Automatic turn-based combat / initiative meter

O que faz. Os combatentes agem automaticamente. A velocidade influencia a frequência das ações, em vez de apenas ordenar uma rodada fixa.

Como foi implementado. ExplorationTurnBattle mantém um medidor por unidade. Quando ninguém está pronto, soma a velocidade dos vivos até alguém atingir 100. Quem age gasta 100 pontos. Empates consideram medidor, velocidade, equipe e nome. O controlador apresenta as ações com intervalo de 0,65 s.

```
enquanto ninguemTem100: iniciativa += maximo(1, velocidade)
atacante = escolherProntoPorPrioridade()
iniciativa[atacante] -= 100; executarAtaque()
```

Para entender: Ao longo da luta, velocidade 20 permite aproximadamente o dobro de ações de velocidade 10. O jogador inicia a batalha ou recua; não escolhe cada golpe.

No projeto: Scripts/Exploration/ExplorationTurnBattle.cs; Scripts/Exploration/ExplorationCombatSceneController.cs

14 Formação e proteção da linha de frente

Frontline formation / target priority

O que faz. Coloca os mais resistentes na frente e concentra os ataques no primeiro oponente vivo daquela formação.

Como foi implementado. No início, as equipes são ordenadas pela soma de armadura e resistência mágica, com vida máxima como desempate. Aliados mais resistentes ficam à direita; inimigos, à esquerda. Os alvos são o inimigo vivo mais à esquerda ou o aliado vivo mais à direita. Receber dano não reordena a equipe.

```
resistencia = armadura + resistenciaMagica
ordenarFormacao(resistencia, vidaMaxima)
alvo = primeiroVivoDaLinhaDeFrenteAdversaria()
```

Para entender: Um tanque protege quem está atrás porque a regra de alvo o escolhe primeiro. O espaço visual também inclui sprite, barra inteira e nome, mais a margem do desenvolvedor.

No projeto: Scripts/Exploration/ExplorationTurnBattle.cs; Scripts/Exploration/ExplorationCombatSceneController.cs; Scripts/Exploration/ExplorationCombatantView.cs

FICHAS 15 E 16

Atributos, encontros e saque

15 Atributos, vida e redução de dano

Stat scaling / damage mitigation

O que faz. Diferencia os personagens por vida, velocidade, dano e defesas. Ataques físicos e mágicos consultam proteções diferentes.

Como foi implementado. AdventurerStatsCalculator soma os bônus de vida dos equipamentos e calcula dano por atributos multiplicados pelos escaladores. O motor sorteia um valor entre dano mínimo e máximo. ExplorationCombatCalculator subtrai armadura ou resistência mágica e garante pelo menos 1 de dano.

```
defesa = ataqueMagico ? resistenciaMagica : armadura
danoFinal = maximo(1, danoSorteado - defesa)
vidaAtual = maximo(0, vidaAtual - danoFinal)
```

Para entender: Exemplo: 12 de dano físico contra 5 de armadura causa 7. A soma das duas defesas organiza a formação, mas não é subtraída de cada ataque.

No projeto: Scripts/Aventureiro/AdventurerStats.cs; Scripts/Exploration/ExplorationTypes.cs; Scripts/Exploration/ExplorationTurnBattle.cs

16 Encontros variáveis e tabelas de saque

Constrained random encounters / loot tables

O que faz. Varia o grupo inimigo dentro dos limites do local e sorteia os recursos recebidos. O mesmo sistema de combate serve a vários ambientes.

Como foi implementado. ExplorationEncounterDefinition lista combinações de até três inimigos e filtra pela quantidade e pelo poder permitidos. Sorteia uma combinação válida com uma semente. Cada recompensa possui recurso, chance e intervalo de quantidade. A vitória registra o saque; a entrega ao inventário ocorre no retorno.

```
gruposValidos = filtrarPorQuantidadeEPoder(combinacoes)
inimigos = sortear(gruposValidos, sementeDaMissao)
se sorteioPassaNaChance: quantidade = sortear(minimo, maximo)
```

Para entender: Uma configuração impossível usa a combinação mais próxima e emite aviso. Defesa pode reduzir a faixa de poder conforme o asset do encontro.

No projeto: Scripts/Exploration/ExplorationEncounterDefinition.cs; Scripts/Exploration/ExplorationCombatSceneController.cs; Scripts/Exploration/ExplorationService.cs

FICHAS 17 E 18

Produção e territórios

17 Alocação de trabalhadores e coleta

Worker allocation / passive resource gathering

O que faz. Distribui exploradores por áreas capturadas para gerar recursos em ciclos. É preciso escolher onde empregar a capacidade disponível.

Como foi implementado. O total corresponde ao nível da Construção de Exploradores menos as perdas. O disponível desconta os trabalhadores já alocados. A cada ciclo, o serviço faz um sorteio da tabela por explorador e acumula os resultados no local. Áreas com ataque pendente não produzem.

```
total = maximo(0, nivelDoAlojamento - exploradoresPerdidos)
disponiveis = maximo(0, total - alocados)
porCiclo: sortearRecompensasParaCadaExplorador()
```

Para entender: O intervalo padrão no serviço é de 300 s e pode ser configurado pelo indicador da taverna. Há limite de 100 ciclos processados por chamada de atualização.

No projeto: Scripts/Exploration/ExplorationService.cs; Scripts/Exploration/TavernActionAttentionIndicator.cs

18 Conquista, guarnição e contra-ataque

Territory control / garrison defense

O que faz. Uma região conquistada passa a produzir, mas precisa de proteção. Os contra-ataques criam risco para a expansão e para os exploradores enviados.

Como foi implementado. A vitória marca a área como capturada e agenda um ataque para 300 s depois. O jogador pode deixar de um a três aventureiros de guarda. Ao vencer o prazo, uma guarda permite criar uma missão de defesa; sem guarda, a área é perdida e os exploradores alocados entram na contagem de perdas.

```
aoVencer: capturada = verdadeiro; ataque = agora + 300
aoChegarAtaque: se temGuarda, prepararDefesa()
senao: perderArea(); registrarExploradoresPerdidos()
```

Para entender: A perda sem guarda também limpa o saque pendente do local. Uma defesa pendente interrompe a coleta até a situação ser resolvida.

No projeto: Scripts/Exploration/ExplorationService.cs: CompleteVictory, RefreshAreaThreats e LoseUndefendedArea

FICHAS 19 E 20

Inventário e relatório

19 Inventário e transporte de recursos

Resource inventory / collection and delivery

O que faz. Separa o que está disponível na cidade do que foi coletado e ainda está na região. O jogador precisa transportar a produção dos exploradores para poder utilizá-la.

Como foi implementado. Os materiais são registros com ID e quantidade. pendingWorkerLoot guarda a produção local; TryTransportWorkerLoot soma esses valores ao inventário e limpa a lista pendente. O saque de missão segue outra regra: é entregue ao retornar, com rewardsClaimed impedindo uma segunda entrega.

```
para cada recurso pendente: inventario[id] += quantidade
limparRecursosPendentes(); salvar()
noRetornoDaMissao: entregarSomenteSeAindaNaoRecebido()
```

Para entender: Transportar funciona como uma transferência de valores no código atual; esse botão não inicia uma caravana animada nem um novo prazo de viagem.

No projeto: Scripts/Core/PlayerInventoryService.cs; Scripts/Exploration/ExplorationService.cs

---

20 Relatório de área e retorno visual

HUD feedback / data-driven UI

O que faz. Mostra o estado real do local: recursos, trabalhadores, guarda, vida e próximo ataque. Ajuda o jogador a entender o resultado das suas decisões.

Como foi implementado. ExplorationAreaReportView lê o serviço, instancia linhas com o prefab de item e usa ícone e nome do recurso. Exibe D como produção pendente e T como máximo por ciclo multiplicado pelos exploradores. Liga apenas os slots ocupados e atualiza os textos em intervalos de 0,2 s.

```
D = quantidadeColetadaAguardandoTransporte
T = maximoDoRecursoPorCiclo * exploradoresNoLocal
tempoExibido = minutos(restante) + ":" + segundos(restante)
```

Para entender: No contador de pessoas, “2 / 3” significa 2 no local e 3 ainda disponíveis globalmente. T é um potencial máximo, não uma promessa de saque nem limite de estoque.

No projeto: Scripts/Exploration/ExplorationAreaReportView.cs; Scripts/Exploration/ExplorationRewardItemView.cs

FICHAS 21 E 22

Câmera e profundidade

21 Câmera de acompanhamento e limites

Camera follow / damping / camera confiner

O que faz. Mantém o personagem enquadrado com suavização e restringe a câmera ao mapa. É um sistema de apresentação que afeta a sensação de controle.

Como foi implementado. A cena Centro usa CinemachineFollow para seguir o alvo e CinemachineConfiner2D para respeitar o limite. O Rigidbody2D do jogador tem interpolação. No ajuste atual, o damping horizontal e vertical é 0,15 e o tamanho ortográfico é 5. O chão usa Bilinear para suavizar a amostragem durante o deslocamento.

```
posicaoDesejada = posicaoDoAlvo + deslocamentoDaCamera
camera = aproximarSuavemente(posicaoDesejada)
camera = restringirAoLimiteDoMapa(camera)
```

Para entender: Esses valores descrevem o Centro. Não significam que todas as cenas usem a mesma câmera, nem que exista Pixel Perfect Camera configurada.

No projeto: Scenes/Centro.unity; Prefabs/player.prefab; Images/Map/CentroStaticV3.png.meta

22 Profundidade e transparência por oclusão

Y-sorting / occlusion fading

O que faz. Faz o personagem aparecer à frente ou atrás dos objetos de forma coerente. Certos objetos ficam transparentes quando encobrem o jogador.

Como foi implementado. WorldPrototypeDepthSort calcula a ordem de desenho pela altura dos pés, usando um ponto estável do colisor. Quanto menor o Y, maior a prioridade de desenho. WorldPropOcclusion verifica posição e sobreposição e reduz o alpha do sprite para 0,35 quando o jogador está atrás.

```
ordem = 2000 - arredondar(yDosPes * 100) + ajuste
se jogadorEstaAtras e imagensSeSobrepoem: alpha = 0.35
senao: alpha = 1
```

Para entender: A referência nos pés evita que uma animação mais alta faça o personagem mudar de camada. O efeito depende dos componentes presentes em cada objeto.

No projeto: Scripts/Prototypes/WorldPrototypeDepthSort.cs; Scripts/Prototypes/WorldPropOcclusion.cs

FICHAS 23 E 24

Cenas e salvamento

23 Viagem entre cenas e pontos de entrada

Scene transitions / spawn points

O que faz. Conecta Centro, portões, taverna e combate sem perder o contexto do destino. O personagem aparece no ponto de entrada adequado.

Como foi implementado. SceneTransitionController bloqueia uma segunda transição, escurece a tela, pausa o relógio estratégico e carrega a cena de modo assíncrono. PlayerSpawnController posiciona o jogador em uma âncora. Depois, a tela clareia e a pausa é liberada. A cena de portão reutiliza o contexto do portão escolhido.

```
bloquearNovaViagem(); pausarCampanha(); escurecer()
carregarCenaAsync(destino); posicionarNaAncora()
clarear(); liberarPausa(); liberarNovaViagem()
```

Para entender: O controlador usa DontDestroyOnLoad para continuar existindo durante a troca. A duração padrão de cada etapa do fade é de 0,2 s.

No projeto: Scripts/Navigation/SceneTransitionController.cs; Scripts/Navigation/PlayerSpawnController.cs; Scripts/Navigation/NavigationService.cs

24 Salvamento e estado persistente

Save/load / persistence

O que faz. Mantém dinheiro, construções, exército, aventuras, regiões e outros progressos entre sessões. Compartilha esses dados entre as cenas.

Como foi implementado. GameSession reúne os serviços sobre um GameSaveData. Commit pede a gravação ao repositório. GameSaveRepository transforma os dados em JSON e grava em PlayerPrefs. Ao carregar, normaliza valores, migra formatos antigos e preserva uma cópia de dados inválidos antes de recuperar um estado padrão.

```
dados = repositorio.Load()
servicos = criarServicosUsandoOsMesmosDados(dados)
aoConfirmarMudanca: repositorio.Save(dados)
```

Para entender: É um save local. A chave atual é tcc.game.save.v2; isso não representa sincronização em nuvem nem retomada exata de cada estado interno da luta.

No projeto: Scripts/Core/GameSession.cs; Scripts/Core/Persistence/GameSaveData.cs; Scripts/Core/Persistence/GameSaveRepository.cs

ENTENDENDO AS LIGAÇÕES

Tempo e organização do código

1. RELÓGIO ESTRATÉGICO

CampaignRuntimeDriver acrescenta **Time.unscaledDeltaTime** ao tempo da campanha. Ondas e viagens de tropas usam esse contador. Ele não avança quando a campanha está pausada nem depois do seu encerramento. As transições de cena solicitam e depois liberam essa pausa.

```
se campanhaPodeAvancar:
    tempoEstrategico += tempoDoFrame
    resolverChegadas(); atualizarOndas(); salvarSeNecessario()
```

2. RELÓGIO REAL

IClock / SystemClock fornece o horário em segundos Unix. Impostos, reposição de soldados, validade das ofertas e prazos de exploração consultam esse horário. Ao serem atualizados, podem reconhecer tempo decorrido fora daquela tela ou com o jogo fechado. Isso não significa que toda a simulação rode continuamente em segundo plano.

Detalhe do projeto: na exploração, a atualização verifica ameaças antes da produção. Um ataque pendente pode bloquear a coleta, e uma área sem guarda pode ser perdida antes de calcular ciclos atrasados. Não se deve prometer produção offline irrestrita.

3. A TELA PEDE; O SERVIÇO DECIDE

Um botão chama um método, como TryUpgrade. O serviço valida as regras, altera os dados e chama Commit. A interface lê o resultado ou recebe um evento para redesenhar os textos. Isso evita esconder a regra de compra dentro do desenho do botão.

```
Botao -> Controller -> Service -> GameSaveData
Service -> Commit -> Repository -> JSON em PlayerPrefs
Resultado ou evento -> View -> texto, imagem e botoes
```

4. POR QUE USAR DEFINIÇÕES SEPARADAS

Char_base, BuildingDefinition e ExplorationEncounterDefinition são **ScriptableObjects**: fichas de configuração editáveis no Unity. O estado da partida fica separado. Dois guerreiros podem usar a mesma definição, mas ter IDs de contratação e nomes diferentes.

Fontes: Scripts/Core/GameSession.cs; Scripts/Core/Time/IClock.cs; Scripts/Campaign/CampaignRuntimeDriver.cs; Scripts/Campaign/CampaignService.cs; Scripts/Exploration/ExplorationService.cs.

PARA CONSULTAR ENQUANTO LÊ O C#

Vocabulário e limites atuais

Classe e método. Uma classe reúne responsabilidades; um método executa uma ação. Exemplo: ExplorationService é a classe e TrySendParty é um método.

Variável, bool e enum. Variável guarda um valor. bool significa sim/não. enum lista opções nomeadas, como Travelling, InBattle e Returned.

MonoBehaviour e ciclo de atualização. Componente ligado a um GameObject. Update roda por frame; FixedUpdate acompanha a física; LateUpdate executa depois de Update.

Prefab e Instantiate. Prefab é um modelo reutilizável. Instantiate cria uma cópia, como uma oferta de aventureiro ou uma linha no relatório.

Evento e coroutine. Evento avisa que algo mudou. Coroutine distribui uma sequência ao longo de frames; é usada na apresentação do combate e nas transições.

Snapshot, seed e bounds. Snapshot é uma cópia dos dados naquele momento. Seed é a semente do sorteio. Bounds são limites espaciais usados para medir e posicionar objetos.

O QUE NÃO DEVE SER CONFUNDIDO COM MECÂNICA PRONTA

Traços e mana: Tracos está vazio. mana e ganho_mana existem na definição, mas não encontrei consumo de mana, habilidades ou efeitos de traços no motor de combate analisado.

Equipamentos: o cálculo atual soma HP dos slots e usa os atributos base para o dano. Os campos de força, destreza e inteligência do Item não são aplicados por AdventurerStatsCalculator; não equivale a um sistema completo de equipamento em runtime.

Morte e recuperação: chegar a zero de vida tira a unidade daquela luta. Não identifiquei remoção permanente do aventureiro contratado. A criação de snapshots e das equipes de defesa inicializa a vida máxima.

Outras mecânicas: não identifiquei salto/coyote time, progressão por XP ou árvore de habilidades nos scripts examinados. Esses nomes não foram incluídos como funcionalidades existentes.

Escopo: este documento é uma análise do código e das configurações locais, não uma certificação de todos os fluxos em runtime. Não foram alteradas regras do jogo para produzir o guia.